

Applicant Bio

I am a student working on robotics applications built on the DORA framework. In late 2025, I built a navigation framework using DORA with 11 nodes defined in a single `run.yml`. When the dataflow failed, the runtime logs only reported a generic failure with no indication of which node was responsible. My only option was to create separate YAML files and test each node individually - a slow, manual, and frustrating process. That experience made the absence of proper testing tooling very concrete for me, and it is the direct reason I am applying for this project.

1. Abstract

DORA currently lacks standardized testing infrastructure. When a multi-node dataflow fails, developers receive minimal diagnostic information and must resort to manual, node-by-node isolation to find the root cause. This is time-consuming and scales poorly as dataflows grow. This project proposes building `dora-test-utils`, a Rust crate that provides mock APIs for unit testing individual nodes, reusable source and sink nodes for integration testing, and record/replay helpers for regression testing. The goal is to make testing DORA nodes as straightforward as testing ordinary Rust or Python code, while building directly on top of the existing `integration_testing` module already present in `dora-node-api`.

2. Motivation

In November 2025, I was building a navigation framework on DORA with 11 nodes. When the dataflow failed at runtime, the log output looked like this:

```
[ERROR] dataflow failed
```

No node name. No stack trace pointing to the source. I had to manually extract each node into a separate YAML file and run them one at a time to find the broken one. With 11 nodes, this took hours. This is not an edge case — it is the normal debugging experience for any non-trivial DORA dataflow today. The problem has three layers:

- No way to test a single node in isolation without spinning up a real dataflow
 - No structured way to inject test inputs or capture outputs programmatically
 - No regression mechanism to detect when a code change breaks previously working behavior
- `dora-test-utils` directly addresses all three.

3. Problem Statement

3.1 Node isolation is manual and expensive

Testing a single node requires writing a separate YAML file, wiring up dummy inputs, and running the full DORA daemon. There is no equivalent of a unit test harness. While `DoraNode::init_testing()` exists in `/apis/rust/node/src/integration_testing.rs`, it relies on environment variables and file-based I/O, making it unsuitable for use inside standard `#[test]` functions.

3.2 Error attribution is opaque

When a multi-node dataflow fails, the runtime does not reliably surface which node caused the failure. Developers must bisect manually by creating isolated YAML files for each node.

3.3 No regression safety net

When a node's logic changes, there is no automated way to verify that its output behavior is preserved. Regressions are caught only when the full system breaks.

3.4 Onboarding friction

New contributors cannot easily write tests for their nodes, which discourages contributions and makes code review harder.

4. Project Goals

Goal 1 - Mock API and unit test harness

Extend the existing `init_testing()` infrastructure to allow a node to be driven with synthetic inputs programmatically, without starting the DORA daemon.

Goal 2 - Reusable source and sink nodes

Provide ready-made binary nodes that emit test data and capture/assert outputs, usable in real dataflow YAML files for integration testing.

Goal 3 - Multi-node test pipeline examples

Demonstrate how to compose source, target, and sink nodes into a self-contained test dataflow.

Goal 4 - Record/replay regression helpers

Record real dataflow I/O to disk; replay it later to detect behavioral regressions.

Goal 5 - CI/CD templates

Provide GitHub Actions workflow files that run the above tests automatically on every commit. DORA already provides a preliminary testing module. It is located in ``apis/rust/node/src/integration_testing.rs``.

5. Technical Approach

5.1 Existing Foundation and Gap Analysis

DORA already provides a preliminary testing module. It is located in `apis/rust/node/src/integration_testing.rs`. It includes:

- Isolated node initialization with ``DoraNode::init_testing()``.
- File-driven I/O using environment variables.
- Basic testing data structures and a compatible serialization format.

However, this mechanism has clear limitations:

- File-driven input injection is available. But a programmatic, type-safe way is missing.
- Single-node execution is supported. Structured testing of multi-node pipelines is not.
- There is no assertion mechanism. Output capture and validation are also absent.
- No recording or replay features exist. This makes regression testing difficult.

``dora-test-utils`` is meant to build on this foundation, not to replace it.

The table shows the integration path between ``dora-test-utils`` and the existing codebase.

It lists the work by stage and the corresponding file locations.

5.2 Architecture

Extend `init_testing()`, Location: `apis/rust/node/src/integration_testing.rs`

Add `dora-test-utils` crate, Location: `/libraries/test-utils/`

Add source/sink node binaries, Locations: `/binaries/test-source/` and `/binaries/test-sink/`

Extend the message recording format, Location: `/libraries/message/src/integration_testing_format.rs`

Add CI templates, Location: `/.github/workflows`. All these tasks belong to the MVP phase.

5.3 Core Modules (175h MVP)

A. NodeHarness-Unit Testing Core

DORA nodes receive events via `EventStream::recv()` (defined in `/apis/rust/node/src/event_stream/mod.rs`) and send outputs via `DoraNode::send_output()` (defined in `/apis/rust/node/src/node/mod.rs`). `NodeHarness` bypasses the daemon connection and drives these interfaces directly in memory:

```
// Extends the existing DoraNode::init_testing() approach
```

```

let mut harness = NodeHarness::new() ;
// Inject input (Arrow format, consistent with real nodes)
harness.send_input("image", arrow_data) ;
// Drive the node to process one event
harness.tick() ;
// Capture and assert output
let output = harness.recv_output("label") ;
assert_eq!(output.as_str(), Some("cat"));

```

Unlike the existing `init_testing()`, this requires no files or environment variables and works inside a standard `#[test]` function.

B. MockEventStream / MockOutputSender

These mock the two core types in the DORA node API:

- EventStream - `/apis/rust/node/src/event_stream/mod.rs`
- `DoraNode::send_output()` - `/apis/rust/node/src/node/mod.rs`

The mock versions implement the same interface but replace the daemon connection with in-memory channels:

```

pub struct MockEventStream {
    rx: mpsc::Receiver<Event>,
}
pub struct MockOutputSender {
    tx: mpsc::Sender<(String, ArrayData)>,
}

```

C. TestSourceNode / TestSinkNode-Integration Testing

Drop-in YAML nodes for testing real multi-node dataflows without modifying the nodes under test:

nodes:

```

- id: test_source
  path: dora-test-source
  outputs:
    - frame
  config:
    data_file: tests/fixtures/input.json    # Arrow JSON format
- id: my_nav_node                    # node under test, unmodified
  path: my_nav_node
  inputs:
    frame: test_source/frame
  outputs:
    - cmd
- id: test_sink
  path: dora-test-sink
  inputs:
    result: my_nav_node/cmd
  config:
    expected_file: tests/fixtures/expected_output.json

```

fail_on_mismatch: true

TestSinkNode writes comparison results to a file after receiving all expected outputs; CI checks the exit code.

5.4 Extended Modules (+175h)

D. RecordSession / ReplaySession-Regression Testing

Built on top of the existing message format in
/libraries/message/src/integration_testing_format.rs, and compatible with the
output format of DORA_TEST_WRITE_OUTPUTS_TO:

```
// Record: attach to a real dataflow and intercept all inter-node messages
let session = RecordSession::attach("dataflow.yml").await ;
session.run_until_complete().await ;
session.save("tests/fixtures/run_001.json") ;
// Replay: feed recorded inputs, diff outputs
let result = ReplaySession::load("tests/fixtures/run_001.json").run().await ;
result.assert_no_regression();
```

The serialization format reuses DORA's existing Arrow + JSON conventions, minimizing migration cost.

E. GitHub Actions Templates

```
.github/workflows/dora-test.yml
- name: Run unit tests
  run: cargo test -p dora-test-utils
- name: Run integration tests
  run: dora run tests/integration/dataflow.yml --test-mode
- name: Run regression tests
  run: cargo test --test regression
```

6. Dual-Scope Plan

The project has two scopes. The first is a 175-hour core MVP. The second is a 175-hour extended scope built on top.

6.1 175-Hour MVP (Core)

The goal is a self-contained testing toolkit. It solves the most common pain points.

Deliverables include:

NodeHarness: Drives any node with synthetic inputs inside a standard #[test].

MockEventStream / MockOutputSender: In-memory replacements for daemon-backed types.

TestSourceNode: Emits test data from a file or inline configuration.

TestSinkNode: Captures outputs and asserts against expected values.

2 – 3 example test pipelines: Show real usage patterns.

Documentation: Includes a setup guide, API reference, and usage examples.

6.2 +175-Hour Extended Scope

It adds advanced testing capabilities. Deliverables include:

RecordSession: Records real dataflow I/O to disk.

ReplaySession: Replays recorded inputs and diffs outputs.

Regression test examples: Show how to use record/replay in practice.

GitHub Actions templates: Ready-to-use CI workflows.

Python bindings (stretch): Exposes the harness to Python nodes via PyO3.

7. Integrated Timeline

The timeline depends on the final version. The early stages are the same for both. Community Bonding. Both versions study DORA internals. They also confirm API design with the mentor.

Weeks 1 – 2: Design the NodeHarness API. Scaffold the crate at /libraries/test-utils/.

Weeks 3 – 5: Implement NodeHarness, MockEventStream, and MockOutputSender.

Weeks 6 – 8: Implement the TestSourceNode and TestSinkNode binaries.

Weeks 9 – 10: Write example pipelines and integration tests.

Weeks 11 – 12: Polish documentation. Gather mentor feedback.

After this, the paths diverge.

For the 175-hour version

Weeks 13 – 14 are a buffer before final submission.

For the 350-hour version

Weeks 13 – 14 implement RecordSession.

Weeks 15 – 17 implement ReplaySession and regression examples.

Weeks 18 – 20 work on CI/CD templates and Python bindings (stretch goal).

8. Risks and Mitigation

Four main risks were identified. Each has a mitigation strategy.

Risk1: Mocking DORA's daemon connection is harder than expected.

Mitigation: Start with the public EventStream and DoraNode interfaces. Consult the mentor before touching internal code.

Risk2: Record/replay serialization is complex for binary Arrow data.

Mitigation: Use integration_testing_format.rs as a base. Add a base64 envelope only where necessary.

Risk3: Scope creep in harness design.

Mitigation: Freeze the API after Week 2. Add new features only after the core is stable.

Risk4: TestSinkNode exit code handling varies across platforms. Mitigation: Test on Linux and macOS early.

Use a result file as a fallback.

9. Deliverables and Evaluation Criteria

The project will be considered successful if:

- A developer can write a #[test] function that drives a DORA node with synthetic inputs and asserts its outputs, without starting the daemon

- A multi-node dataflow can be tested end-to-end using TestSourceNode and TestSinkNode in a YAML file

- The live status view displays runtime updates in the browser. A recorded dataflow run can be replayed to detect regressions (350h version)

- Another developer can follow the documentation to run all examples locally

10. Conclusion

The pain I experienced debugging an 11-node navigation dataflow is not unique to me - it is the current default experience for any DORA developer working on non-trivial applications. DORA already has the seeds of a testing infrastructure in integration_testing.rs, but it stops short of what developers actually need. `dora-test-utils` would close that gap by giving developers the same testing primitives they expect from any mature framework. I want to build the tool I wish had existed when I needed it.